



雙平台輪型機器人設計與應用

指導老師：蘇景暉 李炳輝

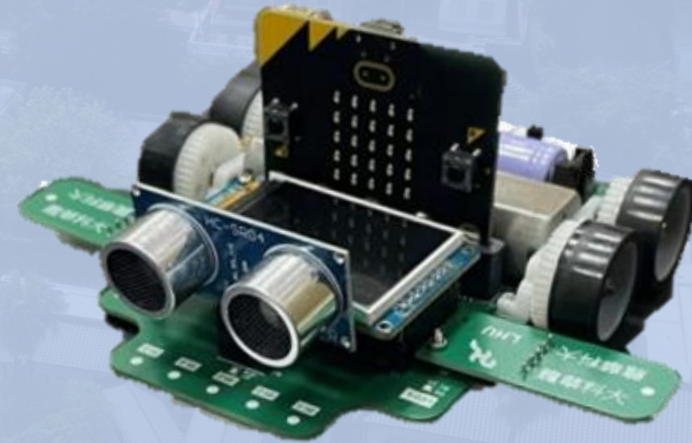


目錄

- 緒論
- 硬體機構設計-機構、電路板
- 線路圖
- 雙平台開發架構
- MakeCode積木程式設計
- Arduino IDE程式設計
- 實際應用與結論

緒論- BitRacer Pro Max 教學輪型機器人

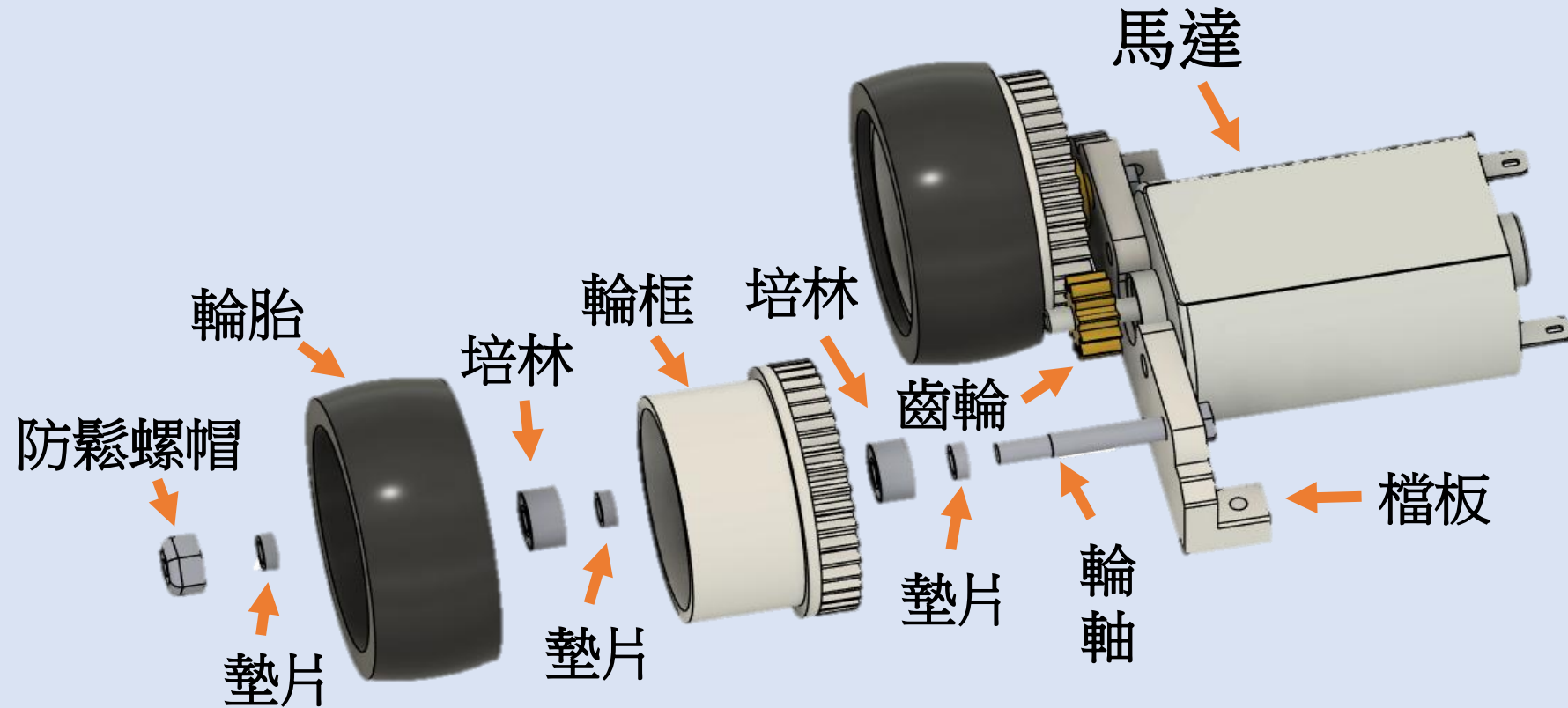
針對108課綱中，關於程式寫作與輪型機器人技術的學習，融入**STEM教育**，由專屬的輪型機器人，來學習智慧輪型機器人等相關實作，並使用「**雙平台開發架構**」提供不同程度學習者使用。



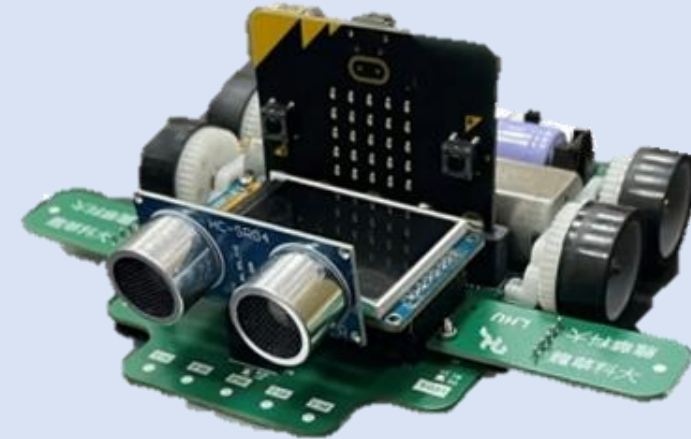
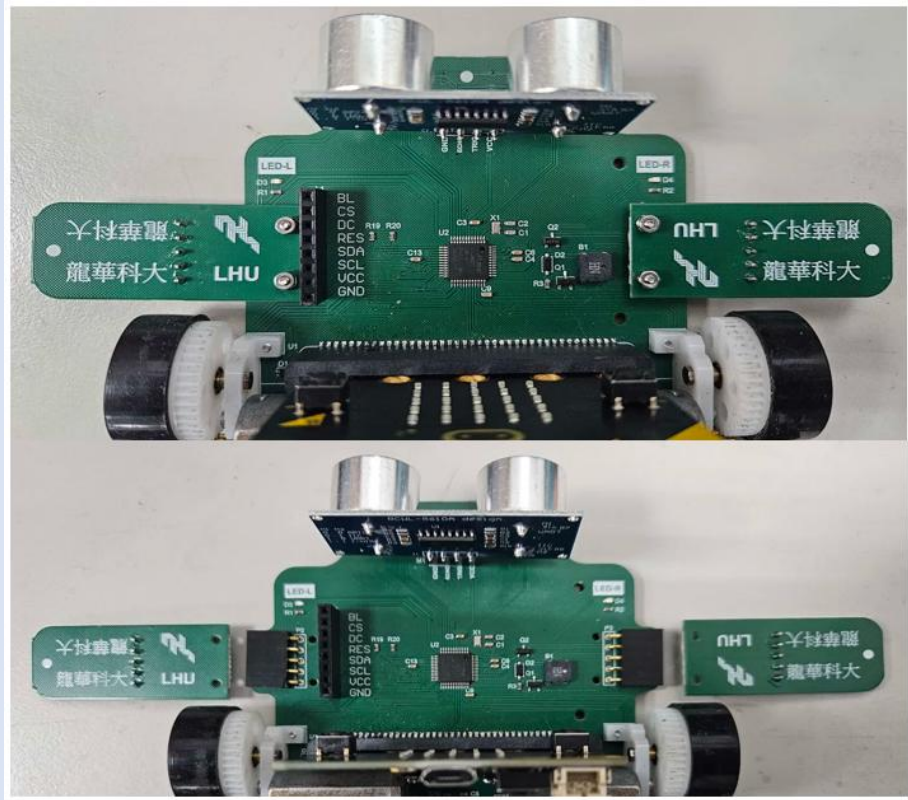
MakeCode



硬體機構設計-馬達四輪



硬體機構設計-紅外線機構





硬體機構設計

電路板設計

BitRacer Pro Max教學輪型機器人

超音波擴充插槽



HC-SR04

紅外線擴展插槽

紅外線感測器 x 5

LED x 2

LCD螢幕顯示插槽

微控制器

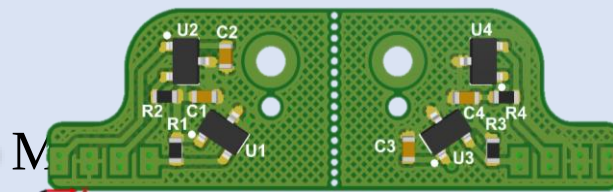
無源蜂鳴器

micro:bit



1.8吋 TFT

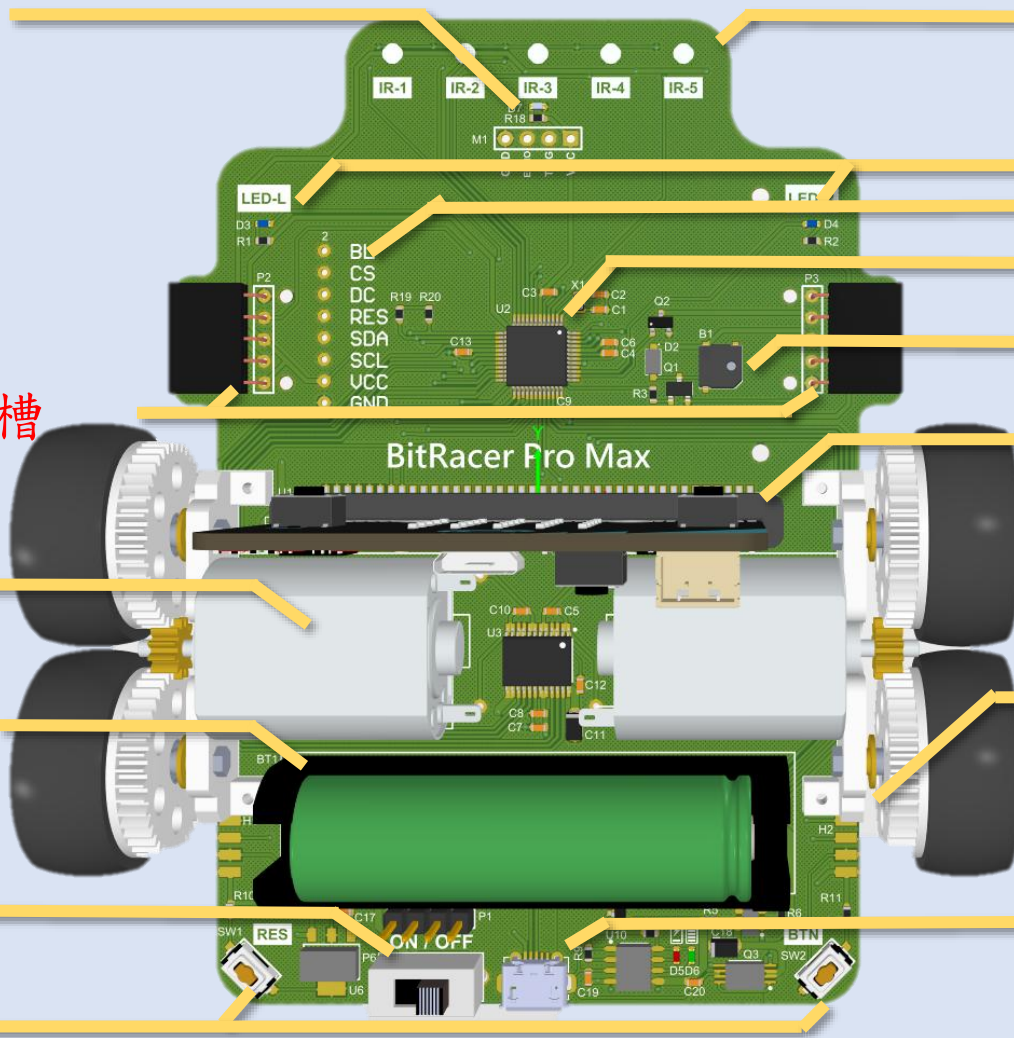
編碼器電路板



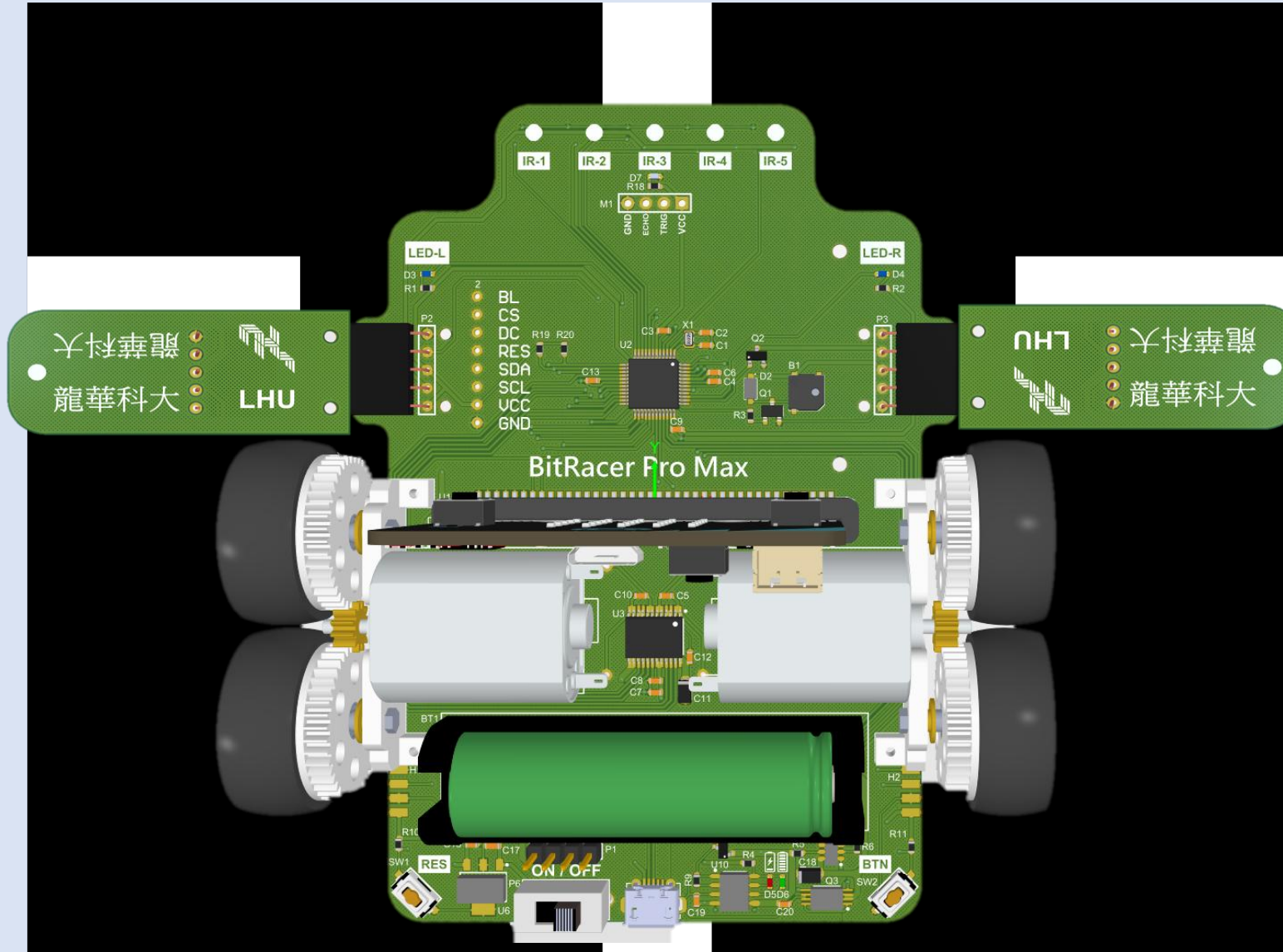
14500鋰電池

電源開關

按鈕 x 2

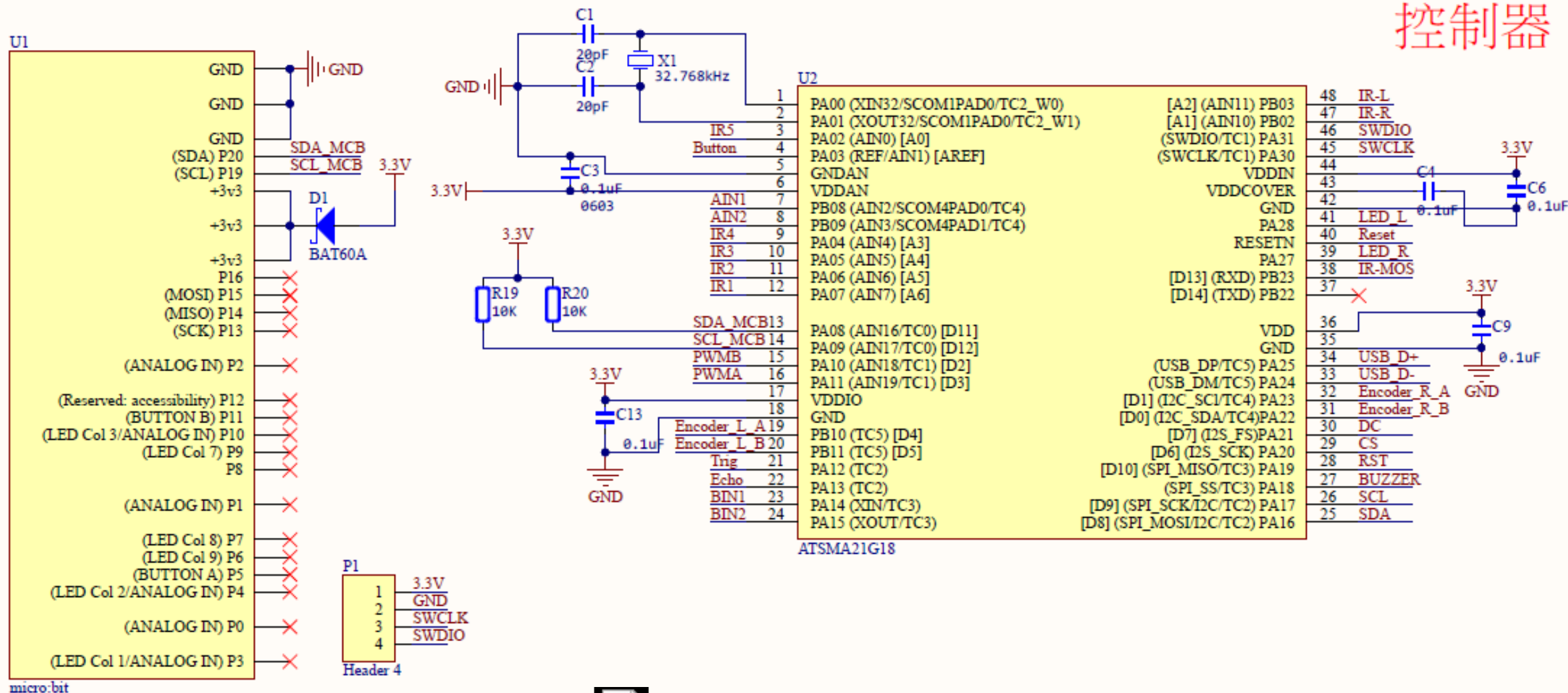


紅外線擴展機構



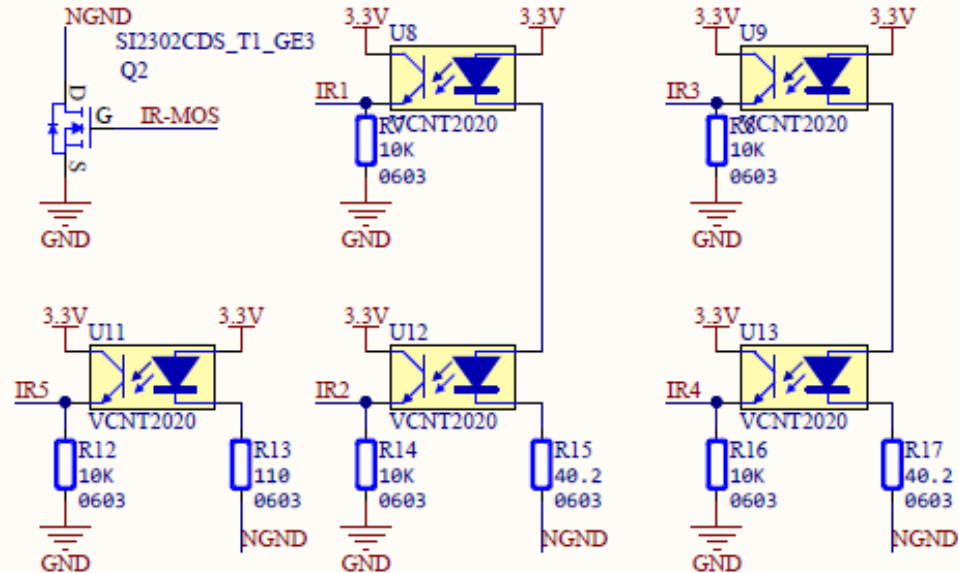
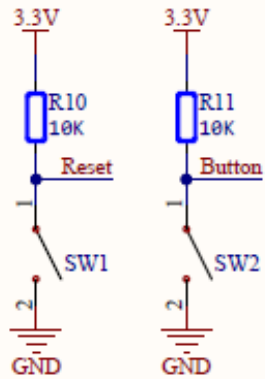
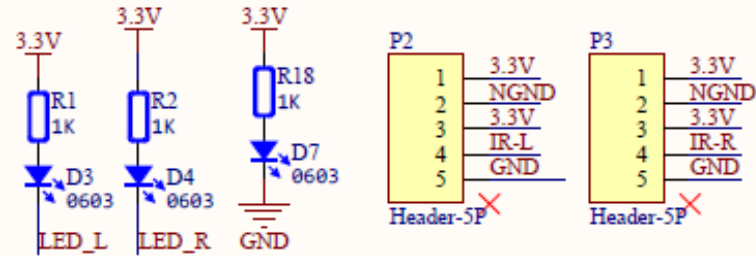
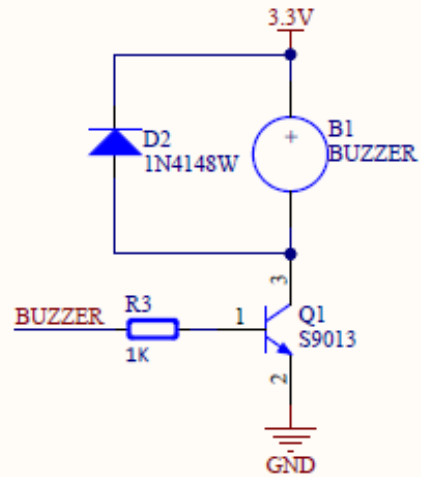
線路圖

控制器



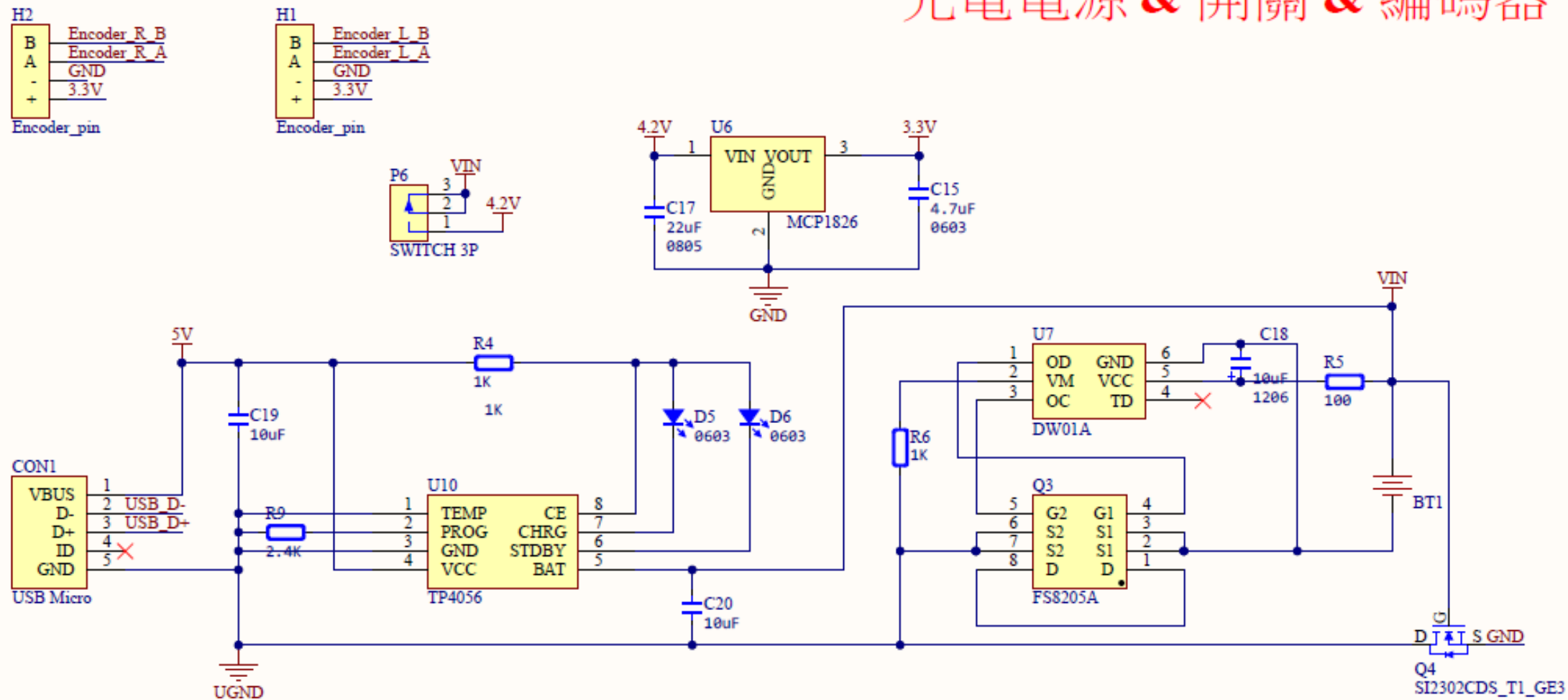
線路圖

紅外線 & 蜂鳴器 & 按鈕LED



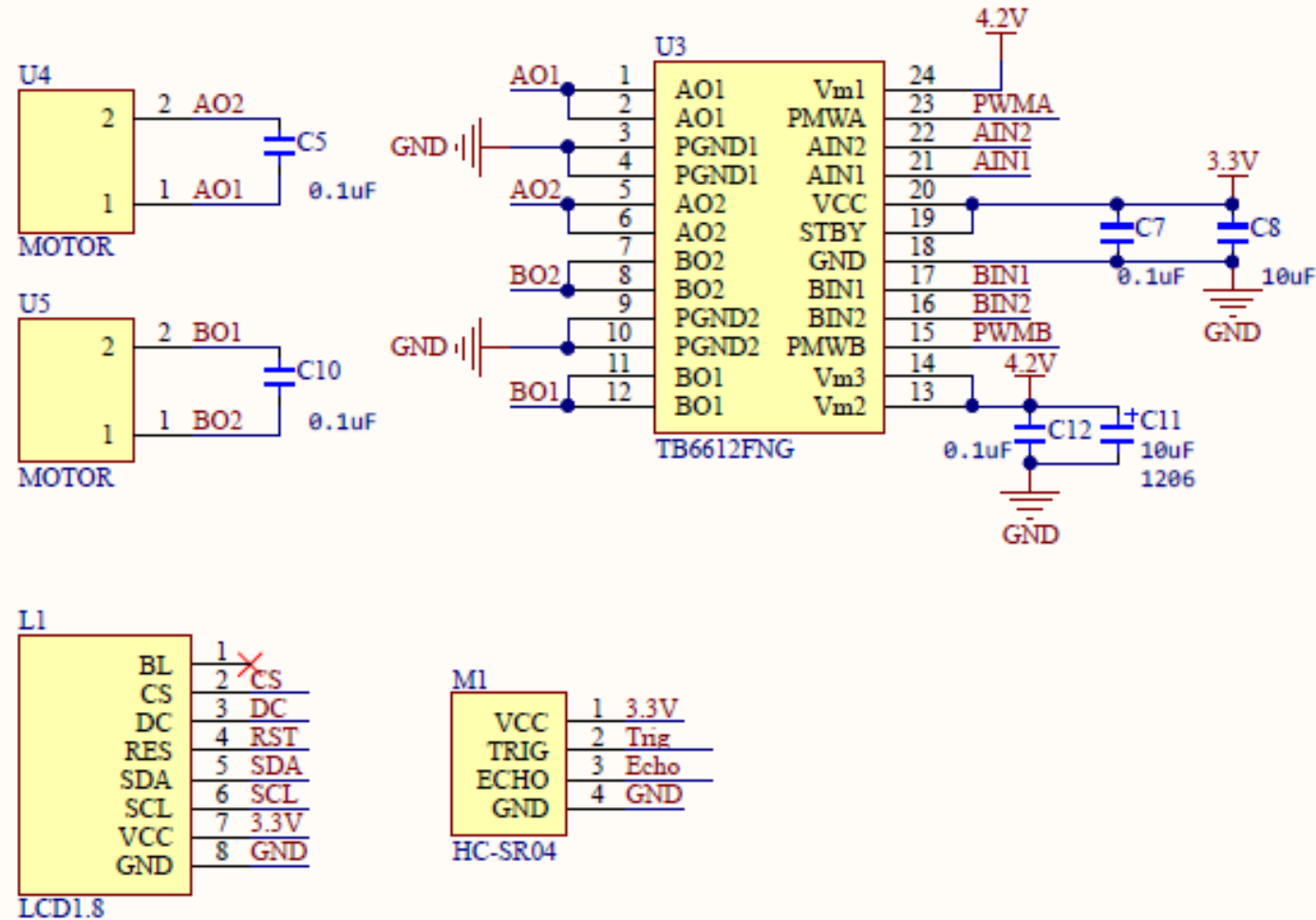
線路圖

充電電源 & 開關 & 編碼器



線路圖

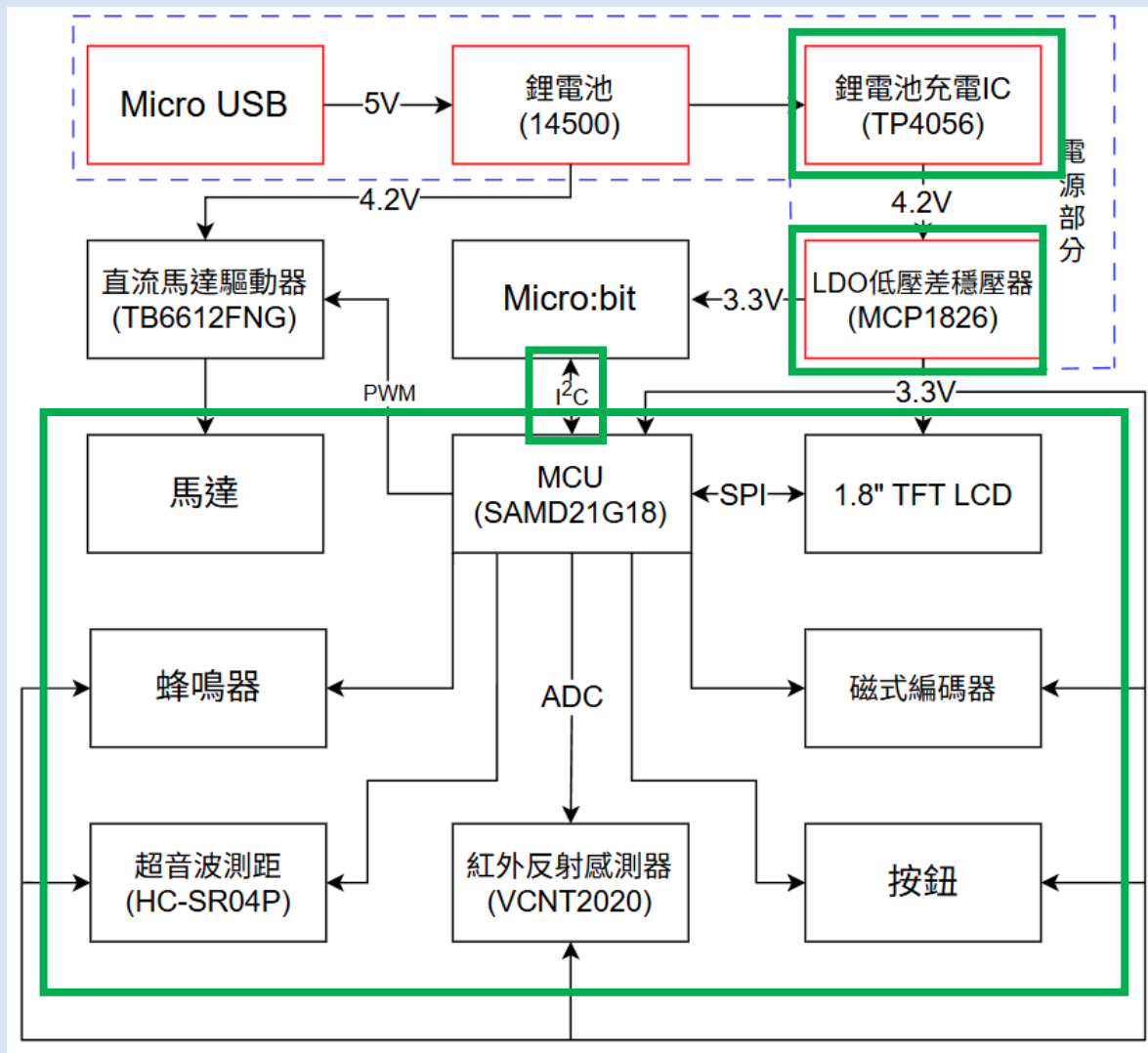
馬達 & LCD & 超音波





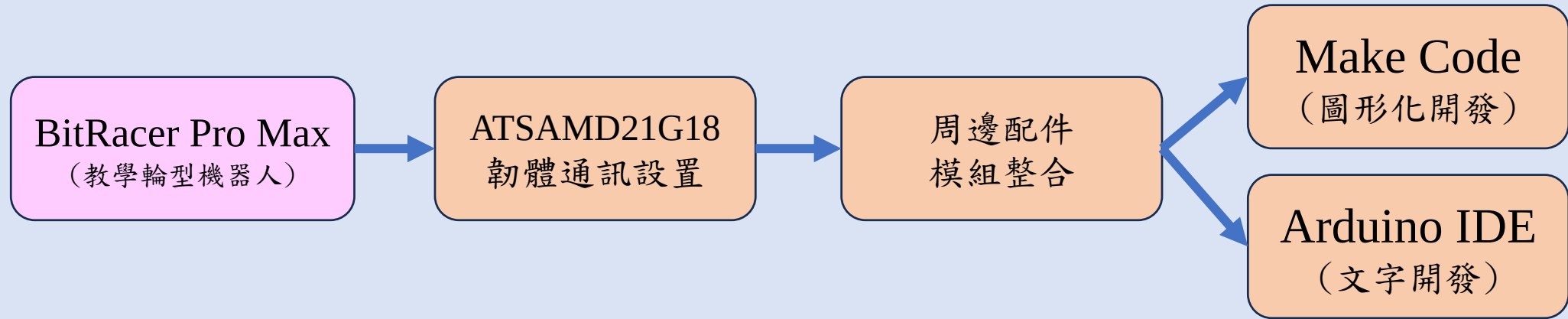
系統架構

BitRacer Pro Max 系統架構



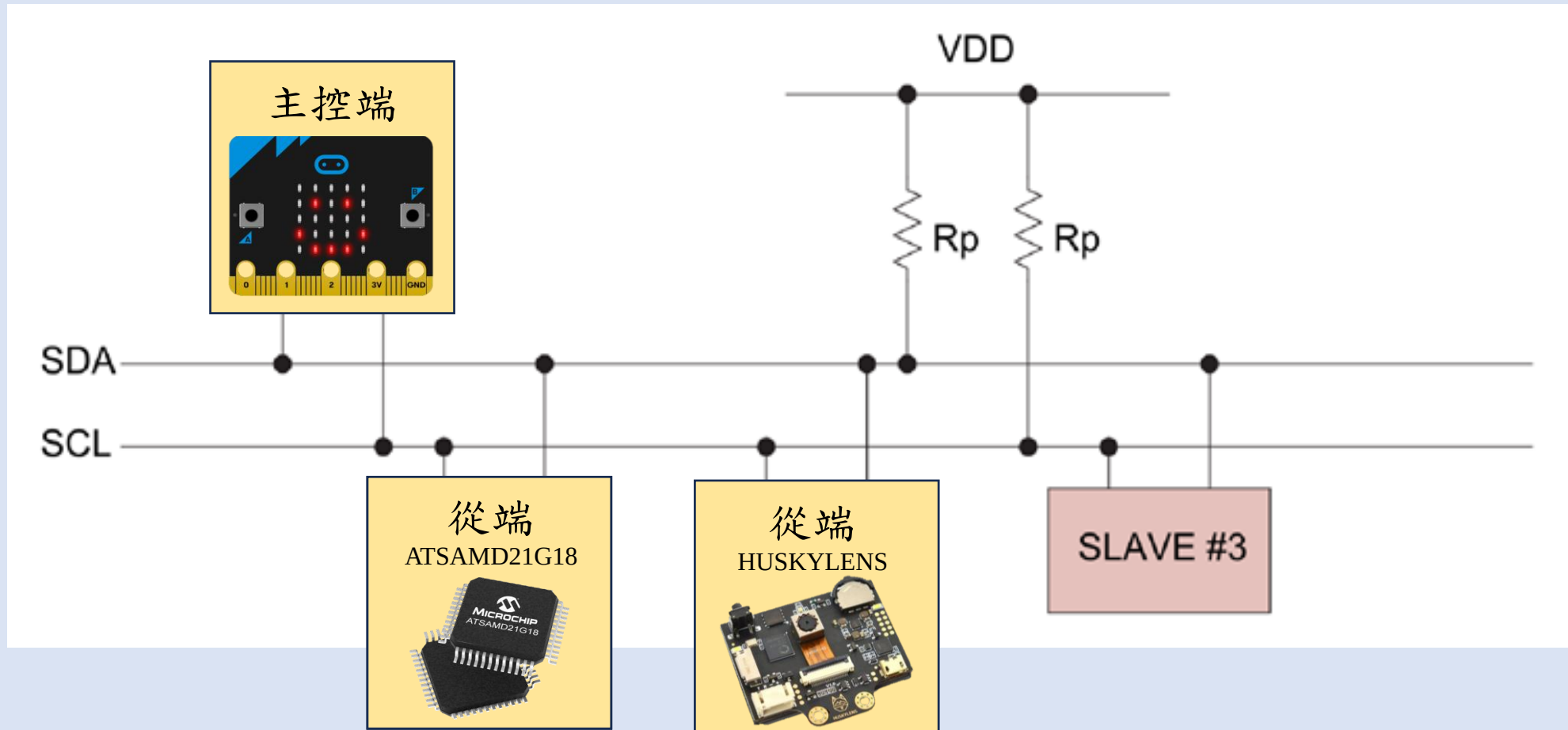


雙平台軟韌體架構

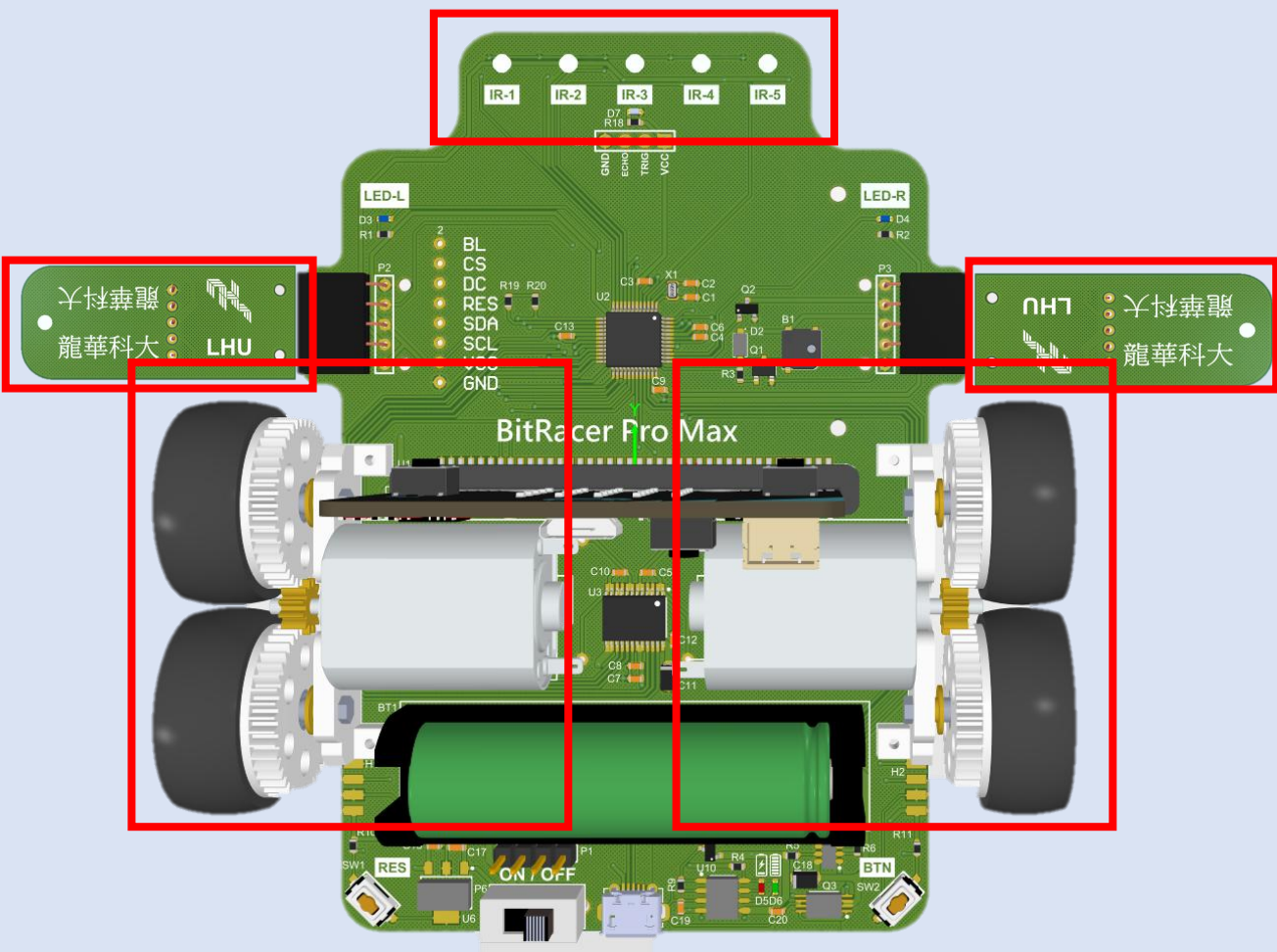


I²C通訊協定

- 透過I²C傳輸資料

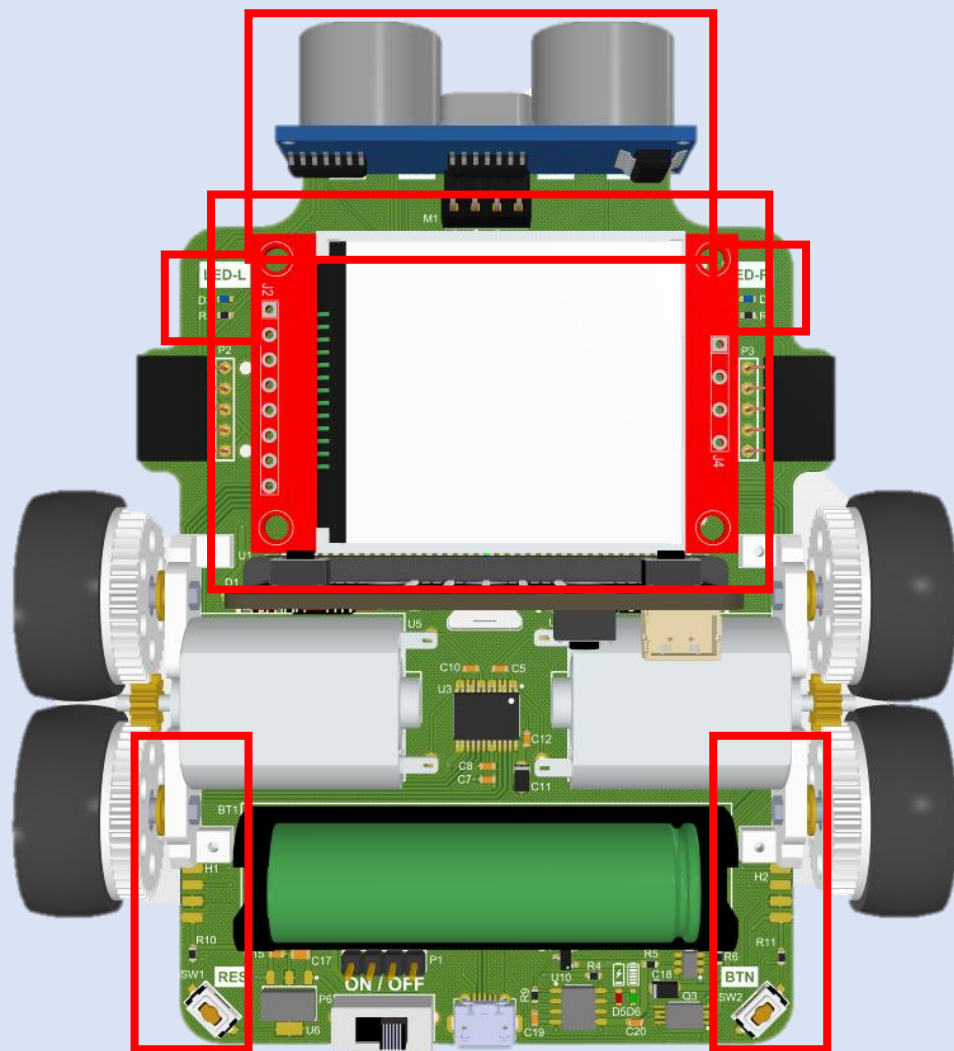


BitRacer Pro Max基礎命令功能



功能	命令編號	讀/寫
馬達左輪	0x20	W
馬達右輪	0x21	W
馬達雙輪	0x22	W
讀取IR-1	0x30	R
讀取IR-2	0x31	R
讀取IR-3	0x32	R
讀取IR-4	0x33	R
讀取IR-5	0x34	R
讀取IR-L	0x35	R
讀取IR-R	0x36	R
控制 IR 開關	0x37	W
讀取所有IR	0x38	R

BitRacer Pro Max 周邊命令功能



功能	命令編號	讀/寫
讀取編碼器數值	0x40	R
編碼器歸零	0x41	W
LED 控制	0x42	W
讀取超音波距離	0x43	R
顯示文字/數字	0x44	W
清除畫面	0x45	W
顯示圖片	0x46	W
設定背景顏色	0x47	W

MakeCode積木單元化設計

- | | |
|---|---------------|
| 1 | 發展環境 & 擴展積木 |
| 2 | 紅外線數值 & 正規化 |
| 3 | 賽道位置數據估測 |
| 4 | 比例差分回授控制 |
| 5 | 超音波距離監控 |
| 6 | 編碼器 & LCD螢幕顯示 |



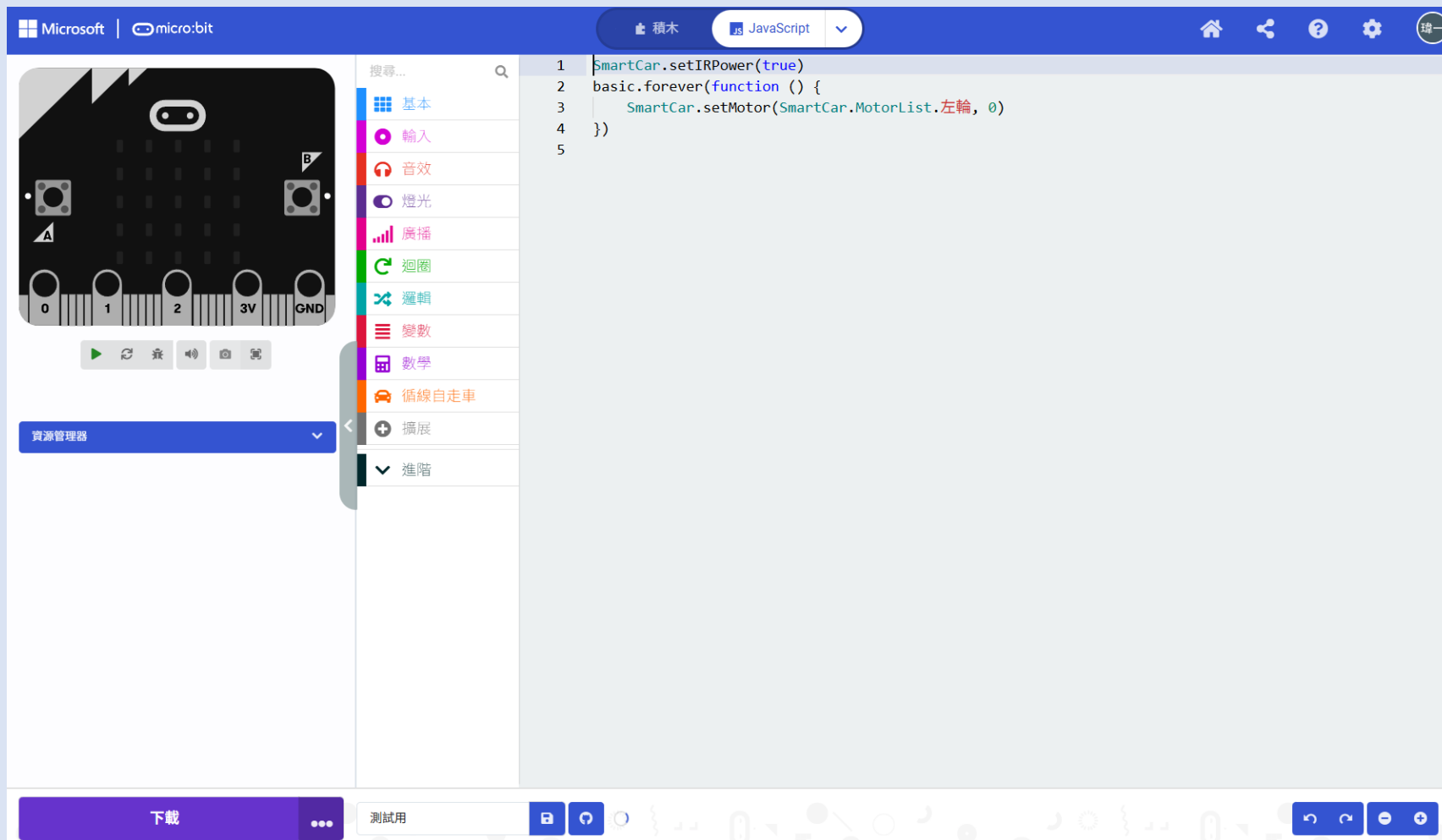


發展環境 & 擴展積木

積木的停車程式

<https://makecode.microbit.org/S92273-39996-50243-0737>

■ 使用Microsoft MakeCode 開發環境



- 製作BitRacer Pro Max專屬擴展積木以簡化通訊



```

//% block="紅外線開關 %on"
//% inlineInputMode=inline on.shadow="toggleOnOff" weight=100 group="感測器控制" color="#008000"
export function setIRPower(on: boolean): void {
    switchBuf[0] = 0x37; switchBuf[1] = on ? 1 : 0
    pins.i2cWriteBuffer(I2C_ADDR, switchBuf, false)
}

//% block="讀取第 %index 顆 感測器"
//% inlineInputMode=inline index.min=0 index.max=6 weight=99 group="感測器控制" color="#008000"
export function getIR(index: number): number {
    pins.i2cWriteNumber(I2C_ADDR, 0x30 + index, NumberFormat.Int8LE, false)
    control.waitMicros(100)
    return pins.i2cReadNumber(I2C_ADDR, NumberFormat.UInt16BE, false)
}

//% block="一次讀取七顆紅外線"
//% weight=98 group="感測器控制" color="#008000"
export function getAllIRValues(): number[] {
    pins.i2cWriteNumber(I2C_ADDR, CMD_IR_ALL, NumberFormat.Int8LE, false)
    control.waitMicros(200)
    let allIrBuf = pins.i2cReadBuffer(I2C_ADDR, 14, false)
    let irArray: number[] = []
    for (let i = 0; i < 7; i++) {
        let val = allIrBuf.getNumber(NumberFormat.UInt16BE, i * 2)
        irArray.push(val)
    }
    return irArray
}
    
```



紅外線數值 & 歸一化

紅外線數值 & 歸一化

- 學習目標:讓學生了解「如何校正不同紅外線感測器間的輸出值」。

$$y_i = \frac{y_{max} - y_{min}}{AD_{i,max} - AD_{i,min}} (AD_i - AD_{i,min}) + y_{min}$$

校正後數值

感測器感測的最大值

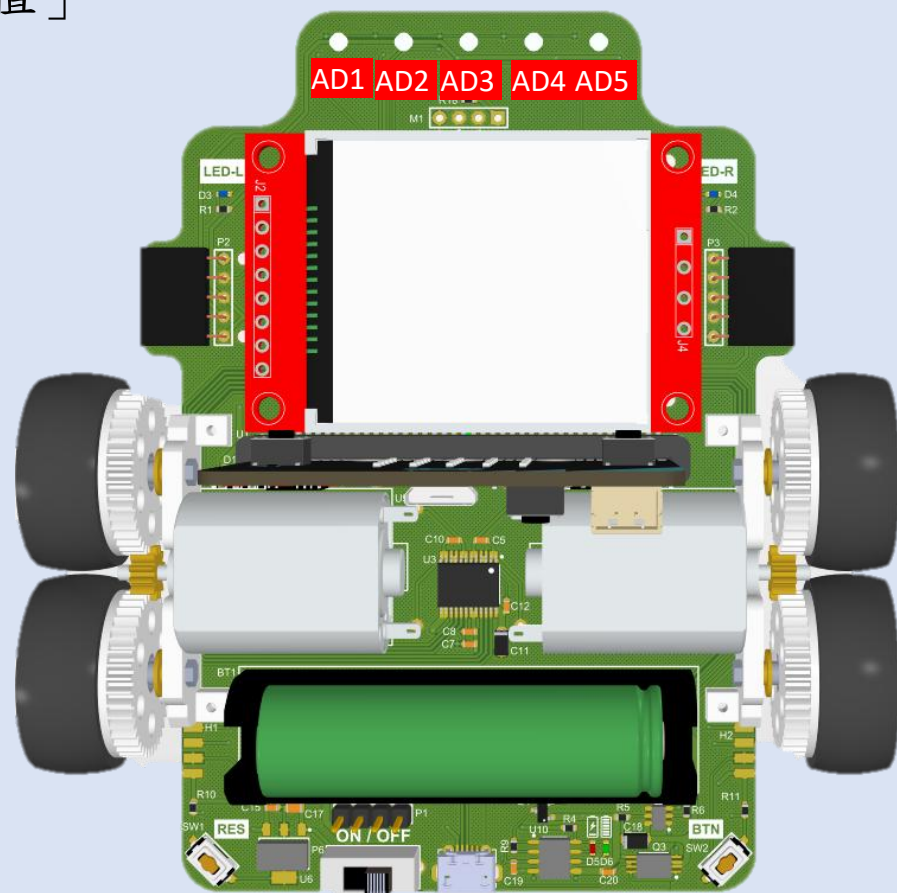
感測器感測的最小值

感測器當下感測數值

預設的最大值

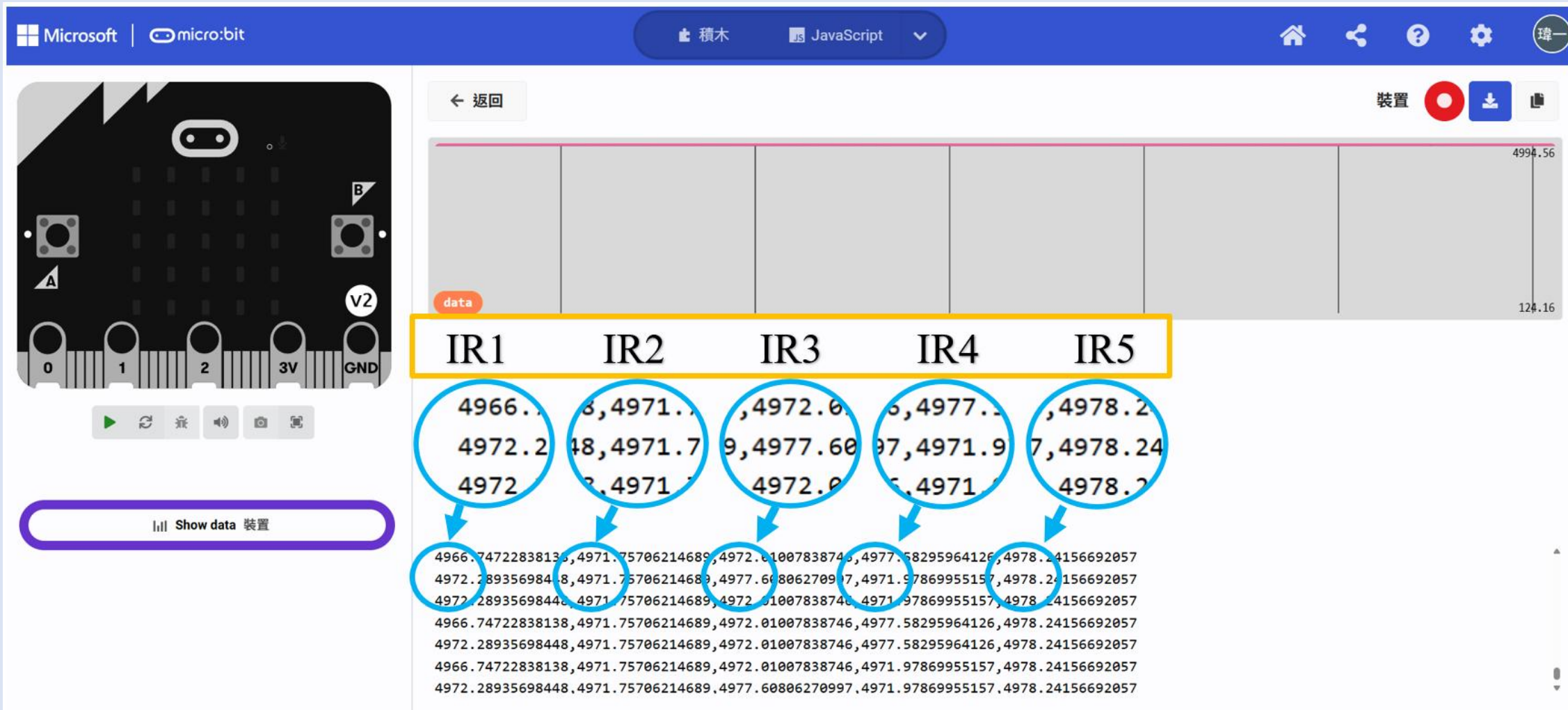
預設的最小值

$$\text{校正後數值} = \frac{5000 - 0}{4000 - 100} (2000 - 0) + 0 \cong 2500$$





紅外線數值 & 歸一化





賽道位置數據估測

賽道位置數據估測

- 學習目標:讓學生們了解「如何利用紅外線數值計算出賽道的位置」。

感測器數值

感測器位置

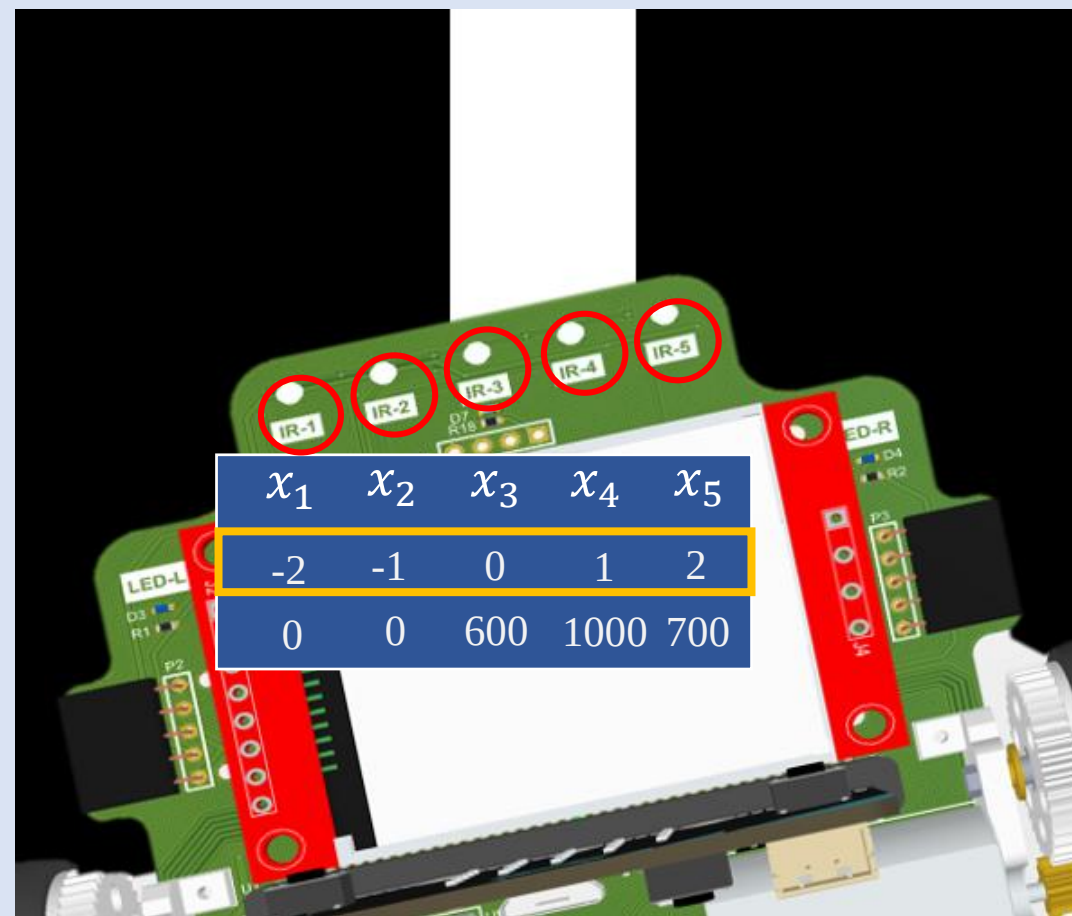
賽道位置數值

$$L_p = \frac{S_1x_1 + S_2x_2 + S_3x_3 + S_4x_4 + S_5x_5}{S_1 + S_2 + S_3 + S_4 + S_5}$$

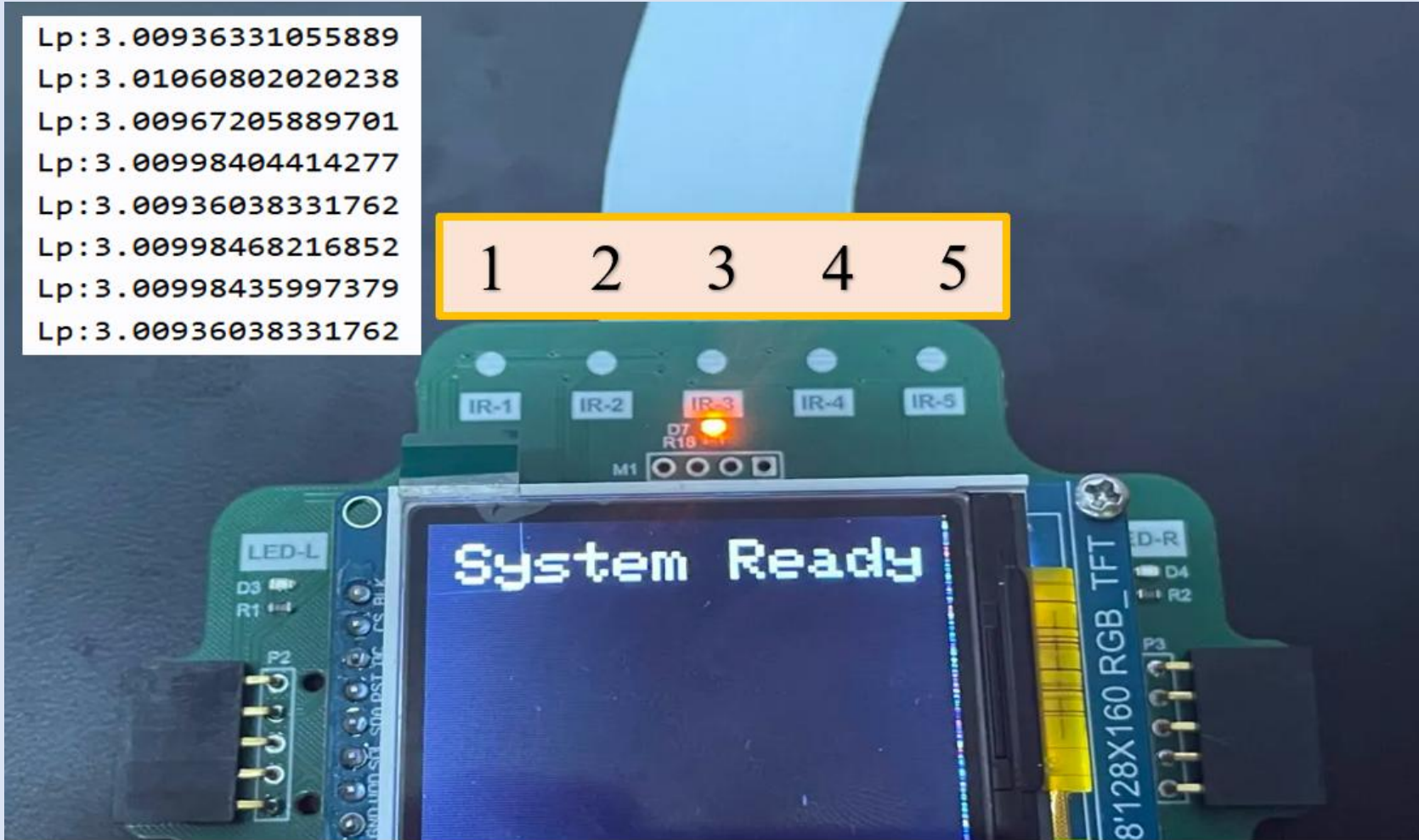
$$L_p = \frac{700 * 2 + 1000 * 1 + 600 * 0 + 0 * -1 + 0 * -2}{700 + 1000 + 600 + 0 + 0}$$

$$= \frac{2400}{2300}$$

$$= 1.0434$$



賽道位置數據估測





比例差分回授控制

比例差分回授控制

- 學習目標:讓學生們了解「了解比例與差分增益的參數，來執行循線目標」。

$$\Delta_{PWM} = K_p e[n] + K_d (e[n] - e[n-1])$$

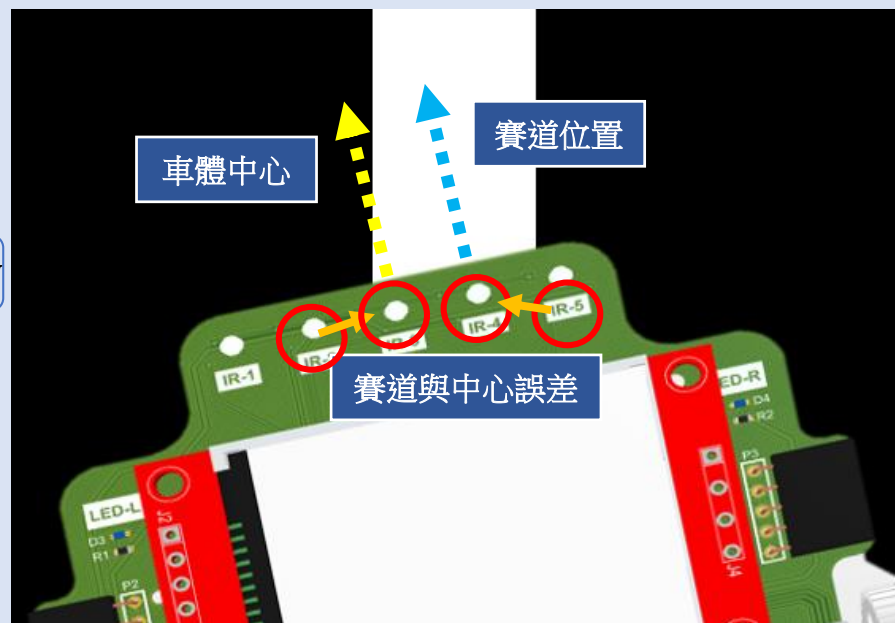
速度控制 與中心誤差 與中心誤差

比例增益 差分增益 前一次誤差

$$PWM_L = PWM_B - \Delta_{PWM}$$

左邊輪子速度 速度控制

基礎速度設計



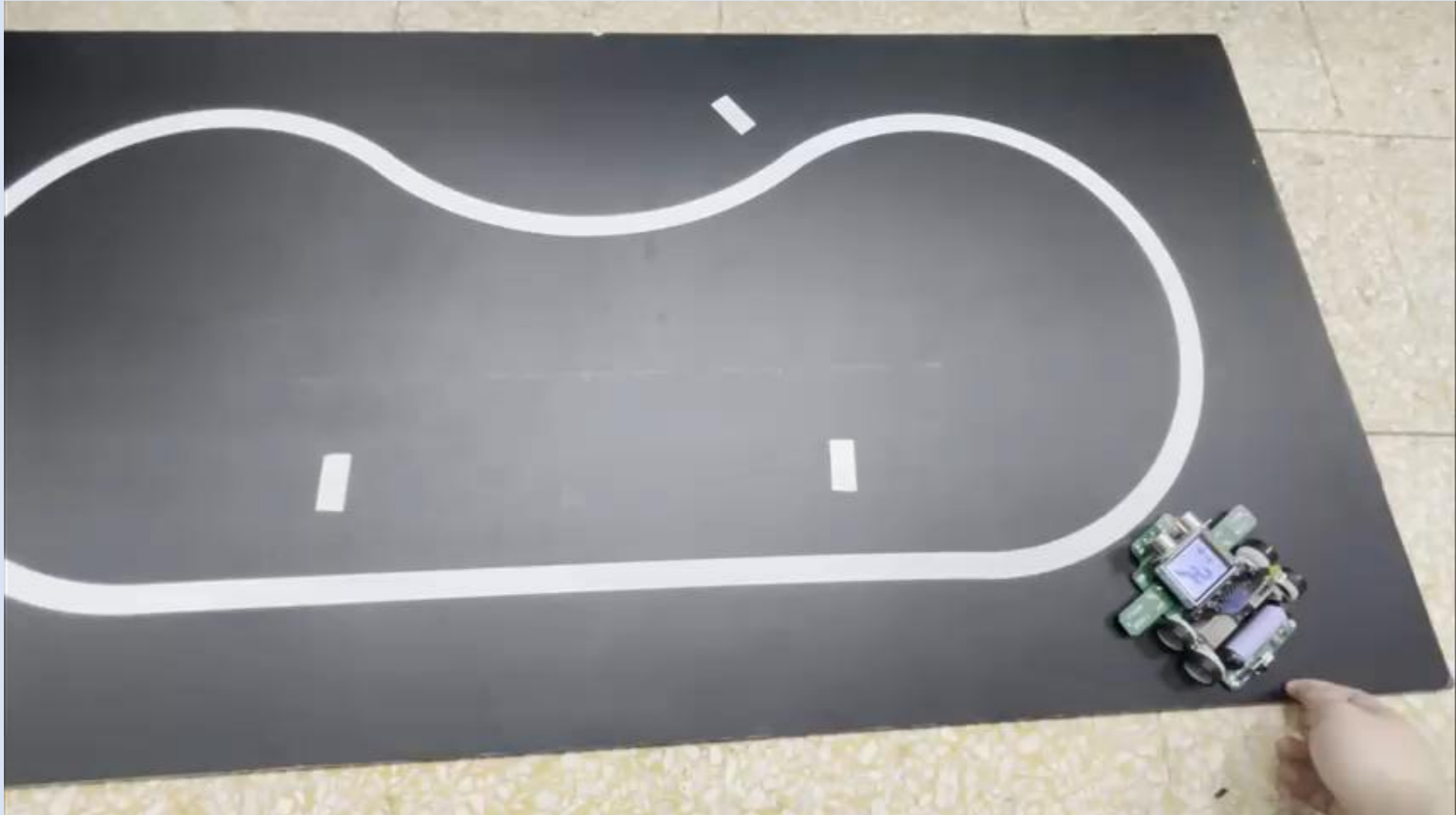
$$PWM_R = PWM_B + \Delta_{PWM}$$

右邊輪子速度 速度控制

基礎速度設計



比例差分回授控制

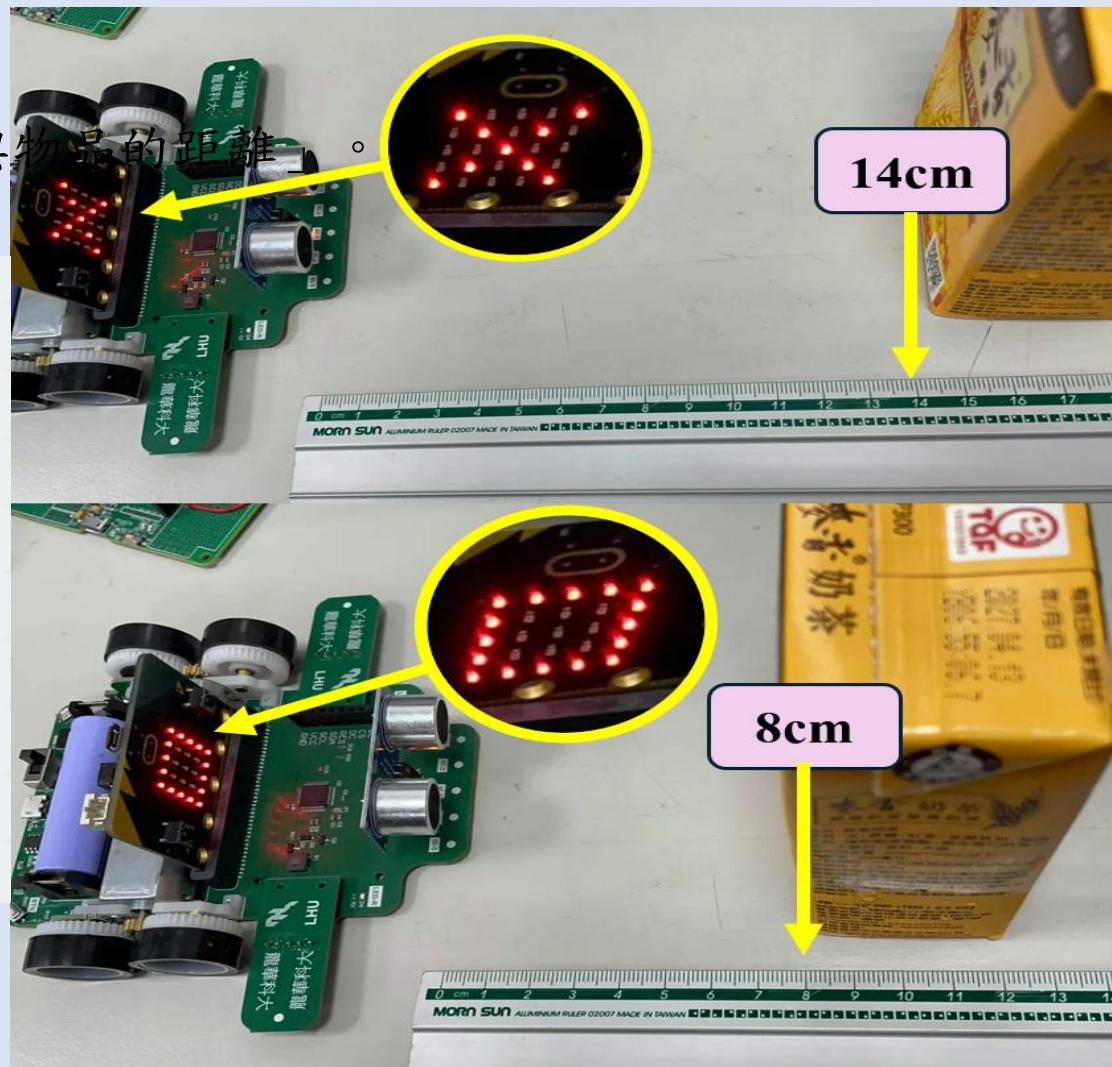




超音波距離監控

超音波距離監控

- 學習目標:讓學生們了解「如何利用超音波計算與物品的距離」。

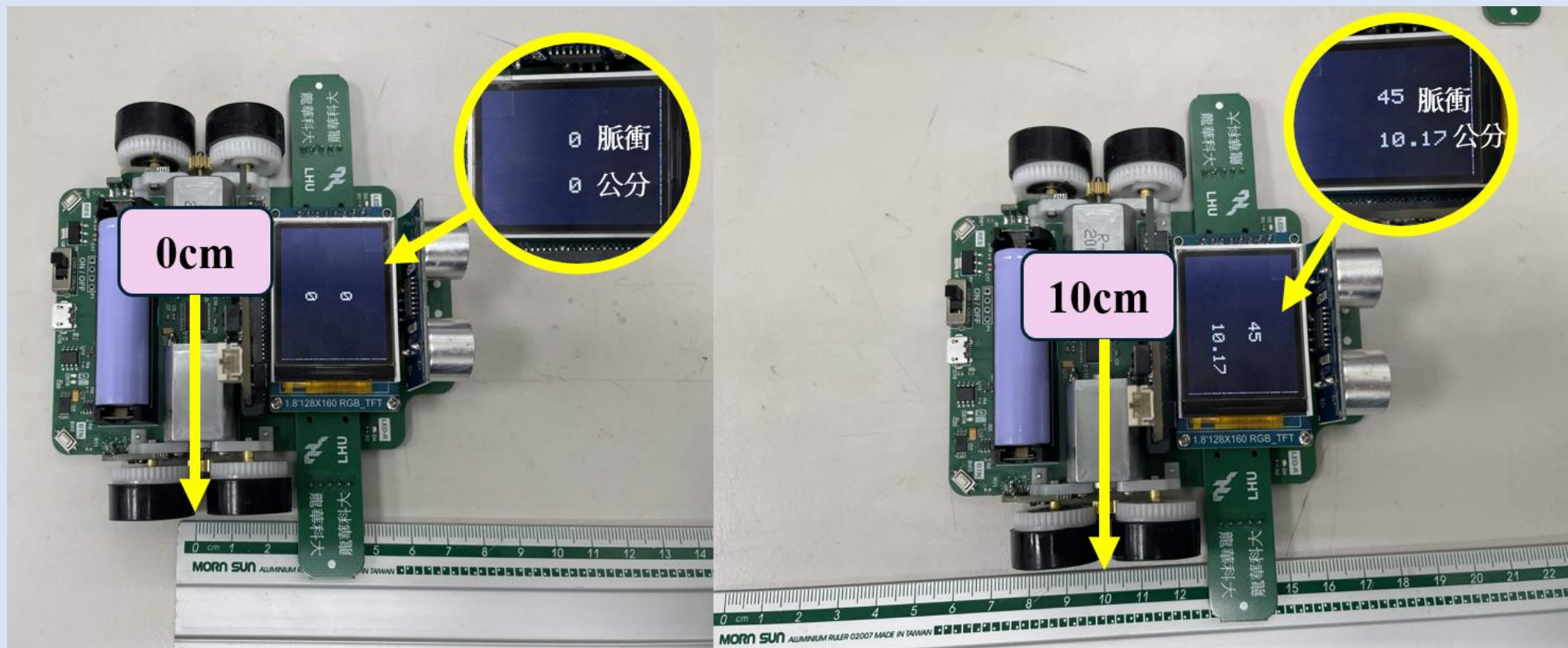




編碼器 & LCD 螢幕顯示

編碼器 & LCD螢幕顯示

- 學習目標:讓學生們了解「如何利用編碼器計算出前進距離，並顯示至LCD螢幕上」。



MakeCode積木程式設計



MakeCode積木程式設計



MakeCode積木程式設計



MakeCode積木程式設計



MakeCode積木程式設計



MakeCode積木程式設計



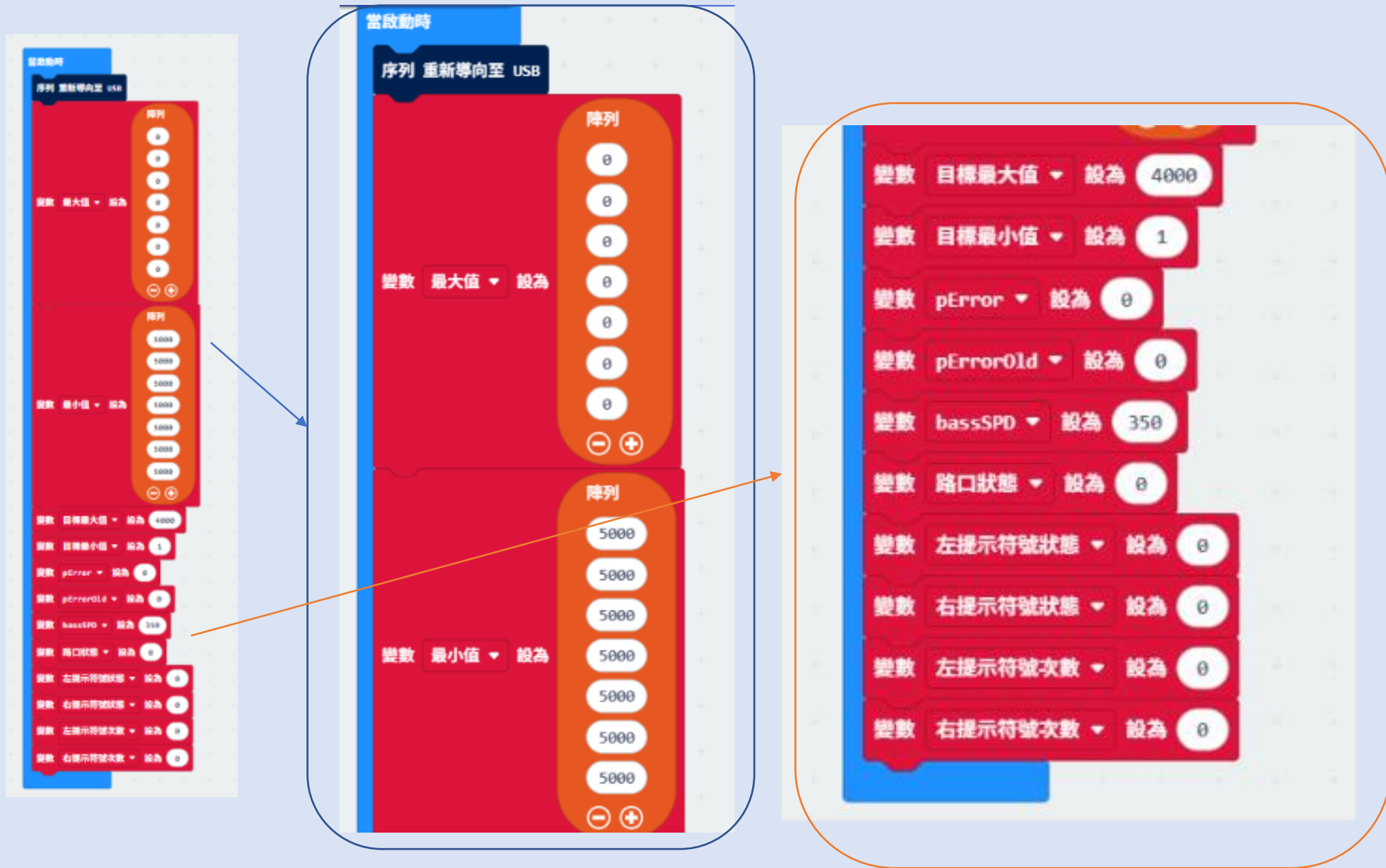
MakeCode積木程式設計



MakeCode積木程式設計



MakeCode積木程式設計



Arduino IDE程式設計

定義腳位

```
//-----紅外線感測器控制-----//
#define IR_1 A6
#define IR_2 A5
#define IR_3 A4
#define IR_4 A3
#define IR_5 A0
#define IR_L A2
#define IR_R A1
#define IRcontrol 13
//-----LED&BUT控制-----//
#define LED_R 30 //LED燈右邊
#define LED_L 31 //LED燈左邊
#define BUTTON 25 //按鈕
//-----馬達控制-----//
#define PWML 2 //左邊馬達腳
#define BIN_1 28 //左邊馬達正反腳1
#define BIN_2 29 //左邊馬達正反腳2
#define AIN_1 32 //右邊馬達正反腳1
#define AIN_2 33 //右邊馬達正反腳2
#define PWMR 3 //右邊馬達腳
```

定義參數

```
//-----速度PD種控制-----//
#define kp 100 //100
#define kd 200 //200
#define Basic_speed 100
//-----陣列設置-----//
int IR[7] = { 0 }; //紅外線初始值
int IR_Max[7] = { 1, 1, 1, 1, 1, 1, 1 }; //紅外線最大
初始值
int IR_Min[7] = { 1023, 1023, 1023, 1023, 1023, 1023, 1023 };
//紅外線最小初始值
float IR_Nor[7] = { 0 }; //紅外線正規化初
始值
//-----變數設置-----//
float x = 0, err = 0, ekd = 0, errold = 0;
int pd = 0; //L = 0;
int L_Cnt = 0, R_Cnt = 0;
```

初始設定

初始設定

```
void setup() {  
  // put your setup code here, to run once:  
  Serial.begin(9600);    //設定傳輸速率  
  pinMode(IR_1, INPUT);  //設定IR_1為輸入  
  pinMode(IR_2, INPUT);  //設定IR_2為輸入  
  pinMode(IR_3, INPUT);  //設定IR_3為輸入  
  pinMode(IR_4, INPUT);  //設定IR_4為輸入  
  pinMode(IR_5, INPUT);  //設定IR_5為輸入  
  pinMode(IR_R, INPUT);  //設定IR_R為輸入  
  pinMode(IR_L, INPUT);  //設定IR_L為輸入  
  pinMode(LED_R, OUTPUT); //設定左邊LED燈為輸出  
  pinMode(LED_L, OUTPUT); //設定右邊LED燈為輸出  
  pinMode(BUTTON, INPUT); //設定按鈕為輸入  
  digitalWrite(LED_R, HIGH); //設定右邊LED燈關閉  
  digitalWrite(LED_L, HIGH); //設定左邊LED燈關閉
```

```
while (digitalRead(BUTTON) != 0)  
  ;  
  delay(1000);  
  Motor(100, 100);  
  int CONT = 0;  
  while (1) {  
    digitalWrite(LED_R, LOW);  
    digitalWrite(LED_L, LOW);  
    Ir();  
    Maxmin();  
    if (CONT >= 50) break;  
    else CONT++;  
  }  
  digitalWrite(LED_R, HIGH);  
  digitalWrite(LED_L, HIGH);  
  Motor(0, 0);  
}
```


主程式

主程式

```
void loop() {  
  while (digitalRead(BUTTON) == 0) {  
    delay(1000);  
    while (1) {  
      follow();  
      prompt();  
      if (R_Cnt == 2) {  
        follow();  
        delay(400);  
        break;  
      }  
    }  
    while (1) {  
      Motor(0, 0);  
    }  
  }  
}
```

副程式1

```
void follow() {  
  Ir();    //呼叫Ir副程式  
  Nor();   //呼叫正規劃副程式  
  weights(); //呼叫權重副程式  
  Print1();  
}
```

副程式-紅外線控制

副程式2

```
void Ir() //Ir副程式
{
    digitalWrite(IRcontrol, HIGH); //開啟紅外線
    IR[6] = analogRead(IR_L);    //讀取IR_L的數值並儲存在IR陣
    列第6位
    IR[0] = analogRead(IR_R);    //讀取IR_R的數值並儲存在IR陣
    列第0位
    IR[3] = analogRead(IR_3);    //讀取IR_3的數值並儲存在IR陣
    列第3位
    IR[4] = analogRead(IR_4);    //讀取IR_4的數值並儲存在IR陣
    列第4位
    IR[2] = analogRead(IR_2);    //讀取IR_2的數值並儲存在IR陣
    列第2位
    IR[5] = analogRead(IR_5);    //讀取IR_5的數值並儲存在IR陣
    列第5位
    IR[1] = analogRead(IR_1);    //讀取IR_1的數值並儲存在IR陣
    列第1位
    digitalWrite(IRcontrol, LOW); //關閉紅外線
}
```

視窗監控

```
void Print1() //顯示副程式
{

    // Serial.print(IR[1]); //顯示IR陣列第0位的數值
    // Serial.print(" \t "); //顯示空格
    // Serial.print(IR[2]); //顯示IR陣列第1位的數值
    // Serial.print(" \t "); //顯示空格
    // Serial.print(IR[3]); //顯示IR陣列第2位的數值
    // Serial.print(" \t "); //顯示空格
    // Serial.print(IR[4]); //顯示IR陣列第3位的數值
    // Serial.print(" \t "); //顯示空格
    // Serial.print(IR[5]); //顯示IR陣列第4位的數值
    //
    //
    // Serial.print(" Nor: "); //顯示正規劃
    // Serial.print(IR_Nor[1]); //顯示正規劃後IR陣列的第0位的
    數值
    // Serial.print(" \t "); //顯示空格
    // Serial.print(IR_Nor[2]); //顯示正規劃後IR陣列的第1位的
    數值
```

```
// Serial.print(" \t "); //顯示空格
// Serial.print(IR_Nor[3]); //顯示正規劃後IR陣列的第2位的
    數值
    // Serial.print(" \t "); //顯示空格
    // Serial.print(IR_Nor[4]); //顯示正規劃後IR陣列的第3位的
    數值
    // Serial.print(" \t "); //顯示空格
    // Serial.print(IR_Nor[5]); //顯示正規劃後IR陣列的第3位的
    數值
    // Serial.print(" \t "); //顯示空格
```

副程式-最大值與最小值、歸一化

```
void Maxmin() //最大最小值副程式
{
    Ir();          //Ir副程式
    for (int j = 0; j < 6; j++) //迴圈(j開始0; j小於6; j加1)
    {
        if (IR[j] > IR_Max[j]) //如果(IR[j]大於IR[j]_最大)
        {
            IR_Max[j] = IR[j]; //IR[j]_最大等於IR[j]
        }
        if (IR[j] < IR_Min[j]) //如果(IR[j]小於IR[j]_最小)
        {
            IR_Min[j] = IR[j]; //IR[j]_最小等於IR[j]
        }
    }
}
```

```
void Nor() //歸一化副程式
{
    for (int i = 0; i < 6; i++) //迴圈(i開始0; i小於6; i加1)
    {
        IR_Nor[i] = (1023.0 / (IR_Max[i] - IR_Min[i])) * (IR[i] - IR_Min[i]);
        if (IR_Nor[i] > 1023) //如果正規化後的IR[i]大於1023
        {
            IR_Nor[i] = 1023; //規化後的IR[i]等於1023
        }
        if (IR_Nor[i] < 1) //如果正規化後的IR[i]小於1
        {
            IR_Nor[i] = 1; //規化後的IR[i]等於1
        }
    }
}
```

副程式-權重與PD控制

```
void weights() //權重
{
    char mode = 'C';
    if (mode == 'C') {
        if (IR_Nor[1] > 700 || IR_Nor[2] > 700 || IR_Nor[3] > 700 ||
IR_Nor[4] > 700 || IR_Nor[5] > 700)
            x = (IR_Nor[1] * -2.0 + IR_Nor[2] * -1.0 + IR_Nor[3] * 0 +
IR_Nor[4] * 1.0 + IR_Nor[5] * 2.0) / (IR_Nor[1] + IR_Nor[2] +
IR_Nor[3] + IR_Nor[4] + IR_Nor[5]);
        } //(規化後的IR[1]乘-2加規化後的IR[2]乘-1加規化後的IR[3]
乘0加規化後的IR[4]乘1加規化後的IR[5]乘2)除(規化後的IR[1]
加規化後的IR[2]加規化後的IR[3]加規化後的IR[4]加規化後的
IR[5])

        erold = err;
        err = 0 - x;
        ekd = err - erold;
        pd = (kp * err) + (kd * ekd);
        Motor(Basic_speed - pd, Basic_speed + pd);
    }
}
```

副程式-驅動馬達

```
void Motor(int16_t PWM_L, int16_t PWM_R) {  
    if (PWM_R >= 255) PWM_R = 255;  
    else if (PWM_R <= -255) PWM_R = -255;  
    if (PWM_L >= 255) PWM_L = 255;  
    else if (PWM_L <= -255) PWM_L = -255;  
    if (PWM_L >= 0) {  
        digitalWrite(BIN_1, HIGH); //設定左邊馬達正反腳1為1  
        digitalWrite(BIN_2, LOW); //設定左邊馬達正反腳1為0  
        analogWrite(PWML, PWM_L); //設定左邊馬達速度為40  
    } else {  
        digitalWrite(BIN_1, LOW);  
        digitalWrite(BIN_2, HIGH);  
        analogWrite(PWML, abs(PWM_L));  
    }  
    if (PWM_R >= 0) {  
        digitalWrite(AIN_1, LOW);  
        digitalWrite(AIN_2, HIGH);  
        analogWrite(PWMR, PWM_R);  
    }  
}
```

```
else {  
    digitalWrite(AIN_1, HIGH);  
    digitalWrite(AIN_2, LOW);  
    analogWrite(PWMR, abs(PWM_R));  
}  
}
```


副程式-提示符號控制

```
void prompt() {
    static int L_status = 0, R_status = 0, C_status = 0;
    if (C_status == 0 && IR[1] > 600 && IR[3] > 600 && IR[5] > 600)
    {
        C_status = 1;
        digitalWrite(LED_L, LOW);
        digitalWrite(LED_R, LOW);
    } else if (C_status == 1) {
        if (IR[6] > 400) L_status = 1;
        if (IR[0] > 400) R_status = 1;
        else if (L_status == 1 && R_status == 1 && IR[6] < 400 &&
IR[0] < 400) {
            digitalWrite(LED_L, HIGH);
            digitalWrite(LED_R, HIGH);
            L_status = 0;
            R_status = 0;
            C_status = 0;
        }
    }
}
```

```
else {
    if (L_status == 0 && IR[6] > 400) {
        L_status = 1;
        digitalWrite(LED_L, LOW);
    } else if (L_status == 1 && IR[6] < 400) {
        digitalWrite(LED_L, HIGH);
        L_status = 0;
        L_Cnt++;
    }

    if (R_status == 0 && IR[0] > 400) {
        R_status = 1;
        digitalWrite(LED_R, LOW);
    } else if (R_status == 1 && IR[0] < 400) {
        digitalWrite(LED_R, HIGH);
        R_status = 0;
        R_Cnt++;
    }
}
}
```

BitRacer Pro Max 教學輪型機器人 實際應用





BitRacer Pro Max 教學輪型機器人

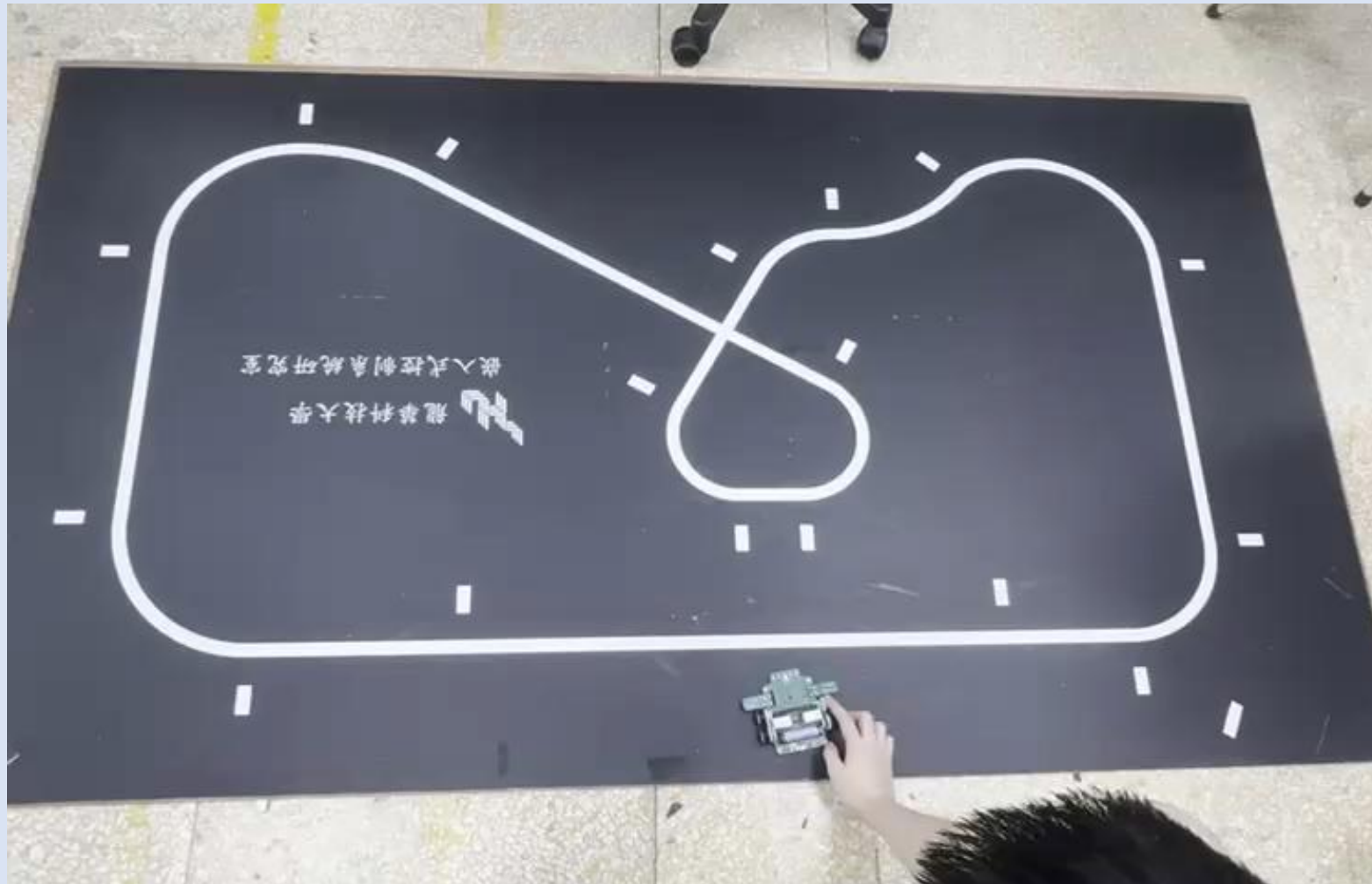
- Microsoft MakeCode





BitRacer Pro Max 教學輪型機器人

- Arduino IDE



BitRacer Pro Max 教學輪型機器人

■ 計算場地總長



BitRacer Pro Max教學輪型機器人，則是以雙平台架構為主並保留了Arduino的硬體與感測器驅動，開發了專屬的MakeCode擴充積木。這使得這台教具『教育的易用性』，讓不同年齡層的學生都能輕鬆跨過學習門檻。

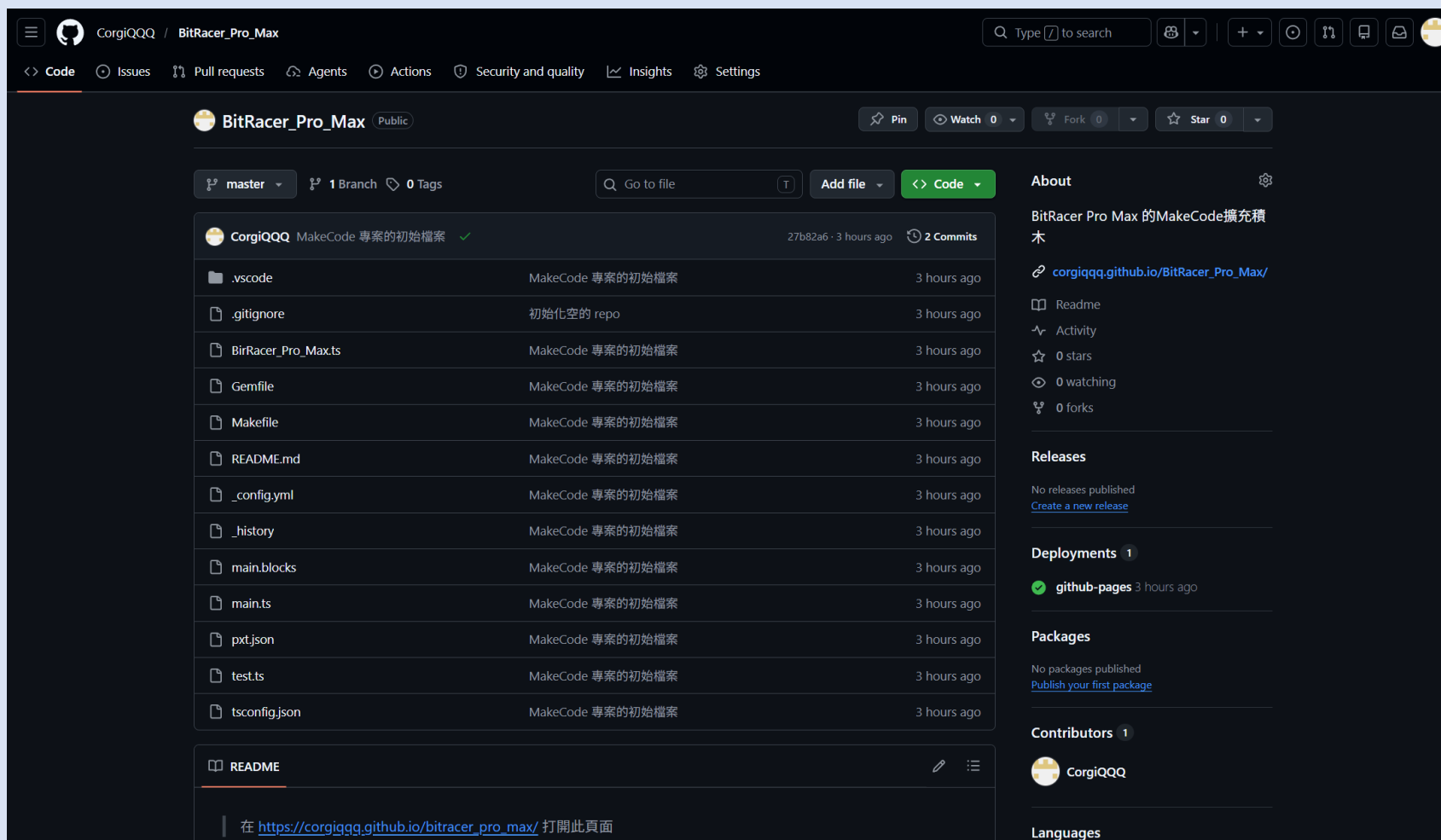




結論

- 將擴展積木分享至Github

https://github.com/CorgiQQQ/BitRacer_Pro_Max



Q & A